Final Report

Advanced Software Product Development:

Internship Assignment Optimization System

Team 3

Hedi Sayadi
Anis Belkacem
Hadir Lakhal
Fekher Salem
Youssef Moustafa

Supervisor: Benedikt Fein

WiSe 2025/26

February 2026

Management Summary

Client and Context

This project was developed for the University of Passau, specifically for the *Zentrum für Lehrkräftebildung und Fachdidaktik* (Center for Teacher Education and Subject Didactics), *Praktikumsamt für Grund- und Mittelschule* (Internship Office for Elementary and Middle Schools).
Each year, this unit is responsible for planning and organizing teaching internships for university students in collaboration with elementary and middle schools throughout the region. This planning process is time-consuming and complex, requiring careful coordination between student requirements, school capacities, and available supervising teachers. The office sought an automated solution to streamline this process while maintaining the quality and feasibility of internship assignments.

Core Challenges

The internship planning process faces several significant challenges:

- **Large number of students:** Hundreds of students require placements each semester, making manual planning impractical
- **Different internship types and subjects:** Multiple types of internships with varying subject requirements must be coordinated
- **Complex constraints:** Numerous rules govern valid assignments, including teacher qualifications, workload limits, and school type matching
- **Distance and accessibility:** Geographic considerations affect both student convenience and teacher availability
- **Overall budget constraints:** The total number of active internship placements is limited by available resources

Solution Delivered

A comprehensive web-based software system that:

- **Automates assignment optimization:** Systematically assigns teachers and students to internship placements while respecting all constraints

- **Multi-phase planning process:** Separates teacher assignment from student assignment, reflecting real-world planning workflow
- **Configurable parameters:** Allows adjustment of budget limits, constraint priorities, and optimization goals
- **Interactive management interface:** Provides tools for data input, assignment review, and manual adjustments when needed

Outcomes

The system delivers tangible benefits to the internship planning process:

- **High-quality, feasible assignment plans:** Produces assignments that satisfy hard constraints while optimizing for quality preferences
- **Customizable solutions:** Maintains flexibility for manual adjustments and what-if scenario planning
- **Reduced manual effort:** Significantly decreases planning time once initial data is entered

Contents

Management Summary **1**

1 Introduction **6**
1.1 Teaching Internships in Teacher Education . . . 6
1.2 Scale and Complexity of Internship Coordination . . . 7
1.3 Limitations of Manual Planning . . . 7
1.4 Course Context . . . 8
1.5 Team Contribution . . . 8

2 Problem Definition & Requirements **10**
2.1 System Vision . . . 10
2.2 Stakeholders . . . 10
2.2.1 Primary: Internship Coordination Unit (*Praktikumsamt*) . . . 10
2.2.2 Secondary: Teaching Students . . . 10
2.2.3 Secondary: Schools and Supervising Teachers . . . 11
2.3 Core Problem Definition . . . 11
2.3.1 Input Data . . . 11
2.3.2 Constraints . . . 12
2.3.3 Objectives . . . 13
2.4 System Characteristics . . . 14
2.4.1 Deterministic Optimization . . . 14
2.4.2 Configurability . . . 14
2.4.3 Interactive Assignment Management . . . 14

3 System Architecture **15**
3.1 Architectural Overview . . . 15
3.1.1 Three-Tier Architecture . . . 15
3.1.2 Design Principles . . . 15
3.2 Technology Stack . . . 15
3.2.1 Backend Technologies . . . 15
3.2.2 Frontend Technologies . . . 16
3.2.3 Development Tools . . . 16
3.3 System Layers and Components . . . 17
3.3.1 Data Access Layer . . . 17
3.3.2 Service Layer . . . 17
3.3.3 Controller Layer . . . 18
3.3.4 Frontend Layer . . . 18
3.4 Core Data Model . . . 18
3.4.1 Entity Relationships . . . 18

3.4.2 Key Attributes . . . 19
3.5 Multi-Phase Assignment Strategy . . . 20
3.5.1 Phase 0: Data Preparation . . . 20
3.5.2 Phase 1: Teacher Assignment and Slot Activation . . . 20
3.5.3 Phase 2: Student Assignment . . . 21
3.5.4 Reoptimization with Baseline Preservation . . . 21

4 Implementation Details **22**
4.1 Repository Structure and Tech Stack (as implemented) . . . 22
4.2 Backend Implementation (Spring Boot) . . . 22
4.2.1 Layering and Main Modules . . . 22
4.2.2 Persistence and Development Database . . . 23
4.2.3 Authentication and Authorization . . . 23
4.3 Optimization Implementation (Two-Phase OptaPlanner) . . . 23
4.3.1 Phase 0: Data Preparation . . . 23
4.3.2 Phase 1: Teacher Assignment and Slot Activation . . . 23
4.3.3 Phase 2: Student Assignment to Activated Slots . . . 25
4.3.4 Reoptimization and Baseline Preservation . . . 26
4.4 Asynchronous Solving and Job Tracking . . . 26
4.5 Import/Export and Validation . . . 26
4.6 Frontend Implementation (React + TypeScript) . . . 27
4.7 How to Run the System Locally . . . 27
4.8 Implementation Notes and Limitations . . . 27

5 Development Process & Results **28**
5.1 Development Methodology . . . 28
5.2 Sprint Overview . . . 28
5.2.1 Sprint 1: Project Foundation . . . 28
5.2.2 Sprint 2: Core Entity Management . . . 28
5.2.3 Sprint 3: Frontend Development and UI Refinement . . . 29
5.2.4 Sprint 4: Optimization Engine Implementation . . . 29
5.2.5 Sprint 5: Refinement and Advanced Features . . . 29
5.3 Team Collaboration . . . 29
5.3.1 Team Structure and Coordination . . . 29
5.3.2 Client Interaction . . . 30
5.3.3 Testing Approach . . . 30
5.4 Key Technical Challenges . . . 30
5.4.1 OptaPlanner ConstraintProvider with JPA Entities . . . 30
5.4.2 Multi-Phase Optimization Architecture . . . 30
5.4.3 Distance Calculation . . . 30
5.5 Performance Evaluation . . . 31
5.5.1 Test Scenarios . . . 31
5.5.2 Results . . . 31
5.6 Known Limitations . . . 32

6 Conclusion **33**
6.1 Summary 33
6.2 Reflection on Optimization Goals 34
6.3 Lessons Learned 34
6.3.1 Technical Insights 34
6.3.2 Process and Collaboration 34
6.4 Future Work 35
6.4.1 Technical Enhancements 35
6.4.2 Feature Extensions 35
6.4.3 Process Improvements 36

Chapter 1

Introduction

1.1 Teaching Internships in Teacher Education

Practical teaching experience is a cornerstone of teacher education programs. While theoretical coursework provides foundational knowledge in pedagogy, child development, and subject-specific didactics, it is through hands-on classroom experience that future teachers develop essential professional competencies. Teaching internships bridge the gap between academic study and professional practice, offering students the opportunity to observe experienced educators, experiment with teaching methods, and receive mentored feedback in authentic school settings.

At the University of Passau, education students pursuing elementary and middle school teaching credentials are required to complete four distinct internship types throughout their program of study:

- **PDP I** (*Pädagogisch-didaktisches Praktikum I*): An initial pedagogical-didactic internship introducing students to classroom observation and basic teaching interactions
- **ZSP** (*Zusätzlich studienbegleitendes Praktikum*): An additional internship accompanying coursework, focusing on subject-specific teaching methods
- **PDP II** (*Pädagogisch-didaktisches Praktikum II*): An advanced pedagogical-didactic internship with increased teaching responsibilities
- **SFP** (*Studienbegleitendes fachdidaktisches Praktikum im Unterrichtsfach*): A subject-didactic internship in a specific teaching subject, integrating theory and practice

Each internship type serves distinct pedagogical goals and occurs at different stages of the degree program. Successful completion of all four internships is mandatory for graduation and licensure as a teacher. Critically, these internships take place in real elementary and middle schools throughout the region, supervised by active classroom teachers who mentor students and evaluate their performance.

1.2 Scale and Complexity of Internship Coordination

The logistical challenge of organizing these internships is substantial. For the 2025/2026 academic year, the *Praktikumsamt* must coordinate approximately **420 planned internship placements** across roughly **100 schools** distributed among **8 regional school districts** (*Schulämter*). Each school employs multiple teachers with varying subject specializations, availability, and capacity to supervise student interns.

The assignment process must account for numerous factors:

- **Student requirements:** Matching each student's internship type, subject specialization, and school type
- **Teacher qualifications and preferences:** Supervising teachers have individual preferences regarding which internship types and subject areas they wish to supervise
- **School characteristics:** Distinguishing between elementary schools (*Grundschule*, GS) and middle schools (*Mittelschule*, MS)
- **Student geographic accessibility:** For PDP I and PDP II internships, students must be assigned to schools close to their home addresses to minimize commute time
- **Capacity constraints:** Respecting internship capacity limits and institutional budget restrictions on the total number of active placements

1.3 Limitations of Manual Planning

Historically, internship assignments have been managed through manual processes involving spreadsheets, email coordination, and institutional knowledge. While this approach has functioned adequately in the past, it presents significant limitations:

- **Time intensity:** Manually matching hundreds of students to appropriate placements requires weeks of dedicated effort by administrative staff
- **Suboptimal assignments:** Without systematic optimization, manual assignments may inadvertently result in long commutes for students, unbalanced teacher workloads, or missed opportunities for better matches
- **Error susceptibility:** Human planners may overlook constraint violations (e.g., assigning a teacher beyond their capacity, mismatching subject requirements) or introduce data entry errors
- **Lack of transparency:** It is difficult to explain why particular assignments were made or to demonstrate that assignments are fair and equitable
- **Inflexibility:** When circumstances change (e.g., a teacher becomes unavailable, enrollment projections shift), revising assignments manually is labor-intensive and may cascade into broader disruptions

- **Limited optimization:** Manual processes struggle to balance competing objectives such as minimizing student travel distance while preserving teacher assignment continuity across semesters

These limitations motivated the *Praktikumsamt* to seek an automated solution that could systematically generate high-quality assignments while reducing administrative burden.

1.4 Course Context

This project was developed as part of the **Advanced Software Product Development (ASPD)** course at the University of Passau during the winter semester of 2025/2026. The ASPD course is structured around real-world client projects, providing students with practical experience in collaborative software engineering.

The course operates with the following organizational structure:

- **Four student teams:** Each team consists of five students, for a total of 20 participants
- **Shared client need:** All four teams address the same problem domain (internship planning), but may approach implementation differently or use different technologies
- **Agile methodology:** Development follows an iterative, sprint-based approach with **five sprints**, each lasting **two weeks**, for a total development period of ten weeks
- **Supervisor guidance:** Each team is guided by a dedicated supervisor who provides mentorship, oversight, and coordination with the client
- **Client engagement:** Communication with the *Praktikumsamt* ensures that requirements are grounded in real needs and that delivered solutions are practically deployable

This structure creates a realistic software development environment where students must balance technical challenges, evolving requirements, team coordination, and stakeholder expectations—mirroring professional software engineering practice.

1.5 Team Contribution

Our team developed a complete, standalone internship assignment system. While all four teams in the course addressed the same client need, each team worked independently to create their own implementation from scratch. Our specific approach and contributions included:

- **Constraint modeling:** Translating the complex business rules governing valid assignments into formal hard and soft constraints
- **Multi-phase optimization architecture:** Designing a phased approach that separates teacher assignment from student assignment to manage computational complexity

- **OptaPlanner integration:** Implementing constraint satisfaction solving using the OptaPlanner framework
- **Score calculation logic:** Developing efficient algorithms to evaluate solution quality based on distance minimization, workload balance, and assignment diversity
- **Backend API development:** Creating RESTful endpoints for data management, solver invocation, and result retrieval
- **Frontend interface:** Building a complete web-based user interface for system interaction

Chapter 2

Problem Definition & Requirements

2.1 System Vision

The internship assignment system is designed to support the *Praktikumsamt's* annual planning cycle while maintaining flexibility and quality. The core vision rests on three pillars:

- **Systematic assignment optimization:** Rather than relying on manual processes, the system applies algorithmic techniques to systematically generate assignments that satisfy constraints and optimize for quality
- **Real-world pragmatism:** Assignments must respect practical constraints including teacher availability, student location preferences, and institutional budgets. The system prioritizes *solution quality* over exhaustive feasibility guarantees, acknowledging that in complex problems with many competing objectives, perfect feasibility may be unattainable
- **Operational transparency:** The system maintains an audit trail of assignments and provides clear justification for decisions, enabling the *Praktikumsamt* to explain assignments to stakeholders and make adjustments when needed

2.2 Stakeholders

2.2.1 Primary: Internship Coordination Unit (*Praktikumsamt*)

The internship office is the primary stakeholder and system user. They require:

- Efficient tools to coordinate hundreds of placements annually, reducing time spent on manual assignment work
- Flexibility to review and manually adjust assignments when business circumstances change

2.2.2 Secondary: Teaching Students

Students require high-quality placements that serve their educational development. Their interests include:

- Geographic accessibility: for PDP I and PDP II internships, assignment to schools near their home addresses to make regular school visits feasible
- Subject-appropriate placements aligned with their intended teaching specialization

2.2.3 Secondary: Schools and Supervising Teachers

Schools and their supervising teachers are the infrastructure that enables internships. Their interests include:

- Balanced workload: not overwhelming individual teachers with too many interns while ensuring adequate supervision
- Alignment with subject expertise and teaching preferences (teachers have preferences for which internship types and subjects to supervise)
- Respect for institutional capacity constraints and available supervision time

2.3 Core Problem Definition

The internship assignment problem can be formally defined by specifying inputs, constraints, and objectives.

2.3.1 Input Data

The system operates on four categories of input:

1. **Students:** Each student record contains:
   - Internship type requirement (PDP I, PDP II, ZSP, or SFP for a given semester/year)
   - Subject specialization (primary teaching subject)
   - Home address (for distance calculations)
2. **Teachers:** Each teacher record contains:
   - Subject expertise and qualifications
   - Preferences: which internship types and subjects they are willing to supervise
   - Maximum internship capacity (workload limit per semester/year)
   - Availability constraints
3. **Schools:** Each school record contains:
   - Type: elementary (*Grundschule*, GS) or middle school (*Mittelschule*, MS)
   - Geographic location (address and coordinates)
   - Zone and public transport accessibility

- List of supervising teachers and their availability

4. **Institutional constraints:** System-level parameters including:
   - Total budget: maximum number of active internship placements for the semester/year

2.3.2 Constraints

The system must respect multiple categories of constraints representing real-world business rules and stakeholder requirements:

Hard Constraints (must be satisfied)

Hard constraints are non-negotiable requirements reflecting institutional policies and operational feasibility:

- **Budget constraint:** The total number of scheduled internship placements cannot exceed the institutional budget limit for the academic year
- **Complete student assignment:** Every student must receive exactly one internship placement matching their internship requirements
- **Teacher workload limits:** Each teacher can supervise 0, 1, 2, or 4 internships per planning cycle (semester or year, depending on internship type).[1]
- **Teacher max workload:** Each teacher can specify a maximum number of internships they prefer not to exceed
- **Internship capacities:** PDP internships can accommodate at maximum two students, SFP and ZSP can accommodate at maximum 4
- **Teacher diversity requirement:** If a teacher supervises multiple internships, they must be of different types. A teacher cannot supervise multiple instances of the same internship type (e.g., two PDP I internships)
- **Subject qualification:** Teachers supervising ZSP or SFP internships (which are subject-specific) must have relevant subject expertise. SFP internships must match the student's main course
- **School type matching:** Students pursuing elementary school certification must be assigned to elementary schools (GS), and those pursuing middle school certification to middle schools (MS)
- **School zones matching:** Internships, by type, must take place in schools with specific zones
- **Teacher availability:** Internship placements can only be scheduled at schools with available, qualified supervising teachers

[1] The *Praktikumsamt_Uebersicht* documentation specifies that teachers should be assigned exactly 2 internships. However, analysis of historical data (example_data) reveals teachers with 1, 2, and 4 internship assignments, with 2 being the most common configuration. The system accommodates this flexibility while treating 2 internships as the preferred workload through soft constraint optimization.

Soft Constraints (preferences to optimize)

Soft constraints are desirable properties that improve assignment quality and stakeholder satisfaction:

- **Student proximity for PDP I & PDP II:** Minimize the geographic distance between assigned students and their home addresses. This ensures practical feasibility for repeated school visits
- **Teacher workload preference:** Prefer assigning teachers exactly 2 internships over other valid counts (1, 4, or 0). This represents ideal teacher engagement
- **Diversity at schools:** Prefer schools with diverse internship types, rather than schools where one type dominates
- **Teacher preference matching:** Favor assignments of internship types and subjects that align with individual teacher preferences
- **Teacher assignment preservation:** When reoptimizing across semesters, prefer preserving teacher assignments for the same types of internships. This promotes continuity and relationship-building between teachers and the university program
- **Subject Matching for ZSP:** Unlike SFP, ZSP internships are flexible regarding the subject course, favor assignments that align with students preferences (main course + 3 additional preferred courses)

2.3.3 Objectives

The system has two primary objectives:

1. **Feasibility:** Generate assignment plans that satisfy all hard constraints. This ensures that assignments are valid and implementable
2. **Optimization:** Among feasible solutions, select plans that maximize soft constraint satisfaction. This means minimizing student travel distances, balancing teacher workloads, respecting preferences, and achieving other quality goals

Importantly, **this system is an optimization system rather than a pure feasibility solver**. In problems with many competing constraints and objectives, guaranteed feasibility across all scenarios may be unattainable. The system prioritizes generating *high-quality, near-feasible* solutions over mathematical certainty of feasibility. This pragmatic approach reflects real-world practice: even if 100% feasibility cannot be assured, the system can produce solutions that satisfy most constraints and provide a high-quality starting point for manual adjustment if needed.

Feasibility Landscape

A key observation mitigates feasibility concerns: the institution has a favorable constraint-to-resource ratio. The total number of available supervising teachers substantially exceeds the number of students requiring placements. For the 2025/2026 academic year, approximately 100 schools with multiple teachers each, must accommodate roughly

420 planned internships. This surplus of teacher availability significantly improves the likelihood of finding feasible solutions in most scenarios.

However, the system is deliberately structured to strategically satisfy constraints even when full satisfaction is impossible, rather than assuming feasibility is guaranteed. This defensive design approach:

- Acknowledges that specific combinations of constraints (e.g., specialized subject requirements + teacher preferences) can create local infeasibility despite overall abundance
- Implements constraint prioritization so that violations, if they must occur, affect lower-priority (preference) constraints rather than critical hard (impossibility) constraints
- Ensures robust behavior even in edge cases or future scenarios with tighter resource availability

2.4 System Characteristics

2.4.1 Deterministic Optimization

The system uses constraint satisfaction and local search optimization to explore the solution space. While it does not guarantee globally optimal solutions (such problems are NP-hard), it consistently produces high-quality solutions within acceptable runtime limits.

2.4.2 Configurability

The *Praktikumsamt* can adjust:

- Budget limits per semester
- Teacher preferences and availability
- Solver time limit (maximum optimization runtime) for phase 1

2.4.3 Interactive Assignment Management

While the core optimization engine operates automatically, the system provides interfaces for:

- Reviewing and validating proposed assignments
- Making manual adjustments when needed (e.g., if a teacher becomes unavailable)

Chapter 3

System Architecture

3.1 Architectural Overview

3.1.1 Three-Tier Architecture

The system follows a classic three-tier architecture separating presentation, business logic, and data persistence:

- **Presentation Tier (Frontend):** Web-based user interface built with React and TypeScript, providing interactive management of students, teachers, schools, and assignments
- **Business Logic Tier (Backend):** Spring Boot application containing optimization engine, constraint validation, and business rules
- **Data Tier:** MySQL relational database for persistent storage of all entities and assignment results

3.1.2 Design Principles

The architecture adheres to several key principles:

- **Separation of concerns:** Clear boundaries between data access, business logic, optimization, and presentation
- **RESTful API:** Stateless HTTP API enables frontend integration and potential future extensions (mobile apps, third-party integrations)
- **Backend-centered optimization:** Computationally intensive constraint solving runs server-side to leverage more powerful hardware
- **Asynchronous processing:** Long-running optimizations execute asynchronously with job status polling to avoid blocking the user interface

3.2 Technology Stack

3.2.1 Backend Technologies

Core Framework:

- **Spring Boot 3.x:** Provides dependency injection, REST controllers, transaction management, and production-ready features
- **Spring Data JPA:** Simplifies database access through repository pattern and object-relational mapping
- **Spring Security:** Handles authentication and authorization for role-based access control

Optimization Engine:

- **OptaPlanner 9.x:** Constraint satisfaction solver implementing local search metaheuristics (Tabu Search, Late Acceptance, Simulated Annealing)
- Embedded within Spring Boot application for tight integration

Database:

- **MySQL:** Relational database used for persistence in the delivered system and development environment
- Supports transactional workloads and reporting queries

Build System:

- **Maven:** Dependency management and build automation

3.2.2 Frontend Technologies

Core Framework:

- **React 18:** Component-based UI library for building interactive interfaces
- **TypeScript:** Type-safe JavaScript providing compile-time error detection and improved IDE support

Build Tools:

- **Vite:** Fast development server and optimized production builds
- Hot module replacement for rapid development iteration

UI Components:

- Custom React components for tables, forms, modals, and visualizations
- CSS modules for scoped styling

3.2.3 Development Tools

- **Git/GitLab:** Version control and code review
- **Docker:** Containerization for consistent development and deployment environments
- **Postman:** API testing and documentation

3.3 System Layers and Components

3.3.1 Data Access Layer

Entities:

- `Student`, `Teacher`, `School`, `Course`: Core master data entities
- `StudentConfig`: Configures which internship types a student needs for a specific semester
- `PlannedInternship`: Represents an internship slot (planning entity for Phase 1)
- `StudentInternshipDemand`: Represents a student's need for a specific internship (planning entity for Phase 2)
- `InternshipAssignment`: Final assignment linking student, teacher, school, and internship type
- `BaselineAssignment`: Captured assignments for semester continuity preservation

Repositories:

- Spring Data JPA repositories for each entity
- Custom query methods for complex lookups (e.g., finding teachers by subject expertise and availability)

3.3.2 Service Layer

Master Data Services:

- `StudentService`, `TeacherService`, `SchoolService`: CRUD operations for master data
- Data validation and business rule enforcement

Optimization Services:

- `Phase1OptimizationService`: Teacher assignment and slot activation
- `Phase2OptimizationService`: Student assignment to activated slots
- `ReoptimizationService`: Semester-to-semester continuity with baseline preservation

Supporting Services:

- `BaselineService`: Captures and retrieves baseline assignments for preservation
- `OptimizationJobService`: Manages asynchronous job status and progress tracking
- `AuditLogService`: Records system actions for transparency and debugging

3.3.3 Controller Layer

RESTful controllers expose endpoints for:

- Master data management (students, teachers, schools, courses)
- Solver invocation (Phase 1, Phase 2, reoptimization)
- Job status polling for asynchronous operations
- Assignment retrieval and export
- System configuration

Example endpoints:

- `POST /api/internships/phase1/optimize-async`: Start Phase 1 optimization
- `GET /api/jobs/{jobId}`: Poll optimization job status
- `GET /api/assignments?schoolYear=WiSe25-26`: Retrieve assignments for a semester

3.3.4 Frontend Layer

Page Components:

- Dashboard: Overview of assignments and system status
- Students, Teachers, Schools: Master data management interfaces
- Assign: Optimization control panel with Phase 1/2/reoptimization triggers
- Audit Logs: System activity history

Shared Components:

- Tables with sorting, filtering, and pagination
- Forms with validation and error handling
- Modals for data entry and confirmation dialogs
- Status indicators for optimization progress

3.4 Core Data Model

3.4.1 Entity Relationships

The data model captures the following key relationships:

- **Student ← StudentConfig:** One-to-many (a student has configurations for multiple semesters)
- **Teacher ← School:** Many-to-one (a teacher works at one school)

- **PlannedInternship → Teacher:** Many-to-one (multiple internships can be supervised by one teacher)
- **PlannedInternship → Course:** Many-to-one (for subject-specific internships)
- **StudentInternshipDemand → StudentConfig:** Many-to-one (multiple demands per student config, one per internship type)
- **StudentInternshipDemand → PlannedInternship:** Many-to-one (assignment relationship determined by solver)
- **InternshipAssignment:** Links student, teacher, school, and internship type in final assignment

3.4.2 Key Attributes

Student:

- Matriculation number (unique identifier)
- Name and contact information
- Home address (geocoded for distance calculations)
- Semester address (if different from home)

Teacher:

- Subject expertise (list of courses they can supervise)
- Preferences (which internship types and subjects they prefer)
- Maximum workload (internship capacity limit)
- Active status (availability toggle)

School:

- Type (Grundschule or Mittelschule)
- Geographic coordinates (latitude/longitude)
- Zone and public transport accessibility (OEPNV classification)
- Active status

PlannedInternship (Phase 1 planning entity):

- Praktikum type (PDP_I, PDP_II, ZSP, SFP)
- School type (GS or MS)
- Active flag (whether this slot is opened)
- Assigned teacher (planning variable)
- Course (planning variable for ZSP, fixed for SFP, null for PDP)

- Capacity (2 for PDP, 4 for ZSP/SFP)

StudentInternshipDemand (Phase 2 planning entity):

- Student configuration reference
- Praktikum type requirement
- Subject preferences (for ZSP course matching)
- Assigned internship (planning variable)
- Baseline internship (for preservation during reoptimization)
- Pinned flag (hard constraint to preserve specific assignments)

3.5 Multi-Phase Assignment Strategy

The assignment process is deliberately structured into multiple phases to manage computational complexity and reflect real-world planning workflows:

3.5.1 Phase 0: Data Preparation

Before optimization begins, the system prepares and validates input data:

- **Demand slot generation:** For each student configuration, create one `PlannedInternship` slot per required internship type
- **Address geocoding:** Convert addresses to coordinates for distance calculations
- **Validation:** Check for missing data, invalid references, and constraint violations
- **Aggregation:** Calculate demand centroids by internship type and school type for geographic optimization

Rationale: Separating preparation from optimization ensures clean, validated input and enables reuse of demand data across multiple optimization runs.

3.5.2 Phase 1: Teacher Assignment and Slot Activation

Goal: Decide which internship slots to activate (within budget) and assign supervising teachers to each active slot.

Planning variables (what OptaPlanner decides):

- For each `PlannedInternship`: `active` (boolean), `assignedTeacher` (Teacher or null), `course` (for ZSP)

Rationale: This phase handles capacity planning (how many internships to offer) and teacher workload balancing before individual student assignment. It reduces complexity by not yet considering individual student preferences.

3.5.3 Phase 2: Student Assignment

Goal: Assign each student to an activated internship slot, minimizing distance and respecting subject preferences.

Planning variables:

- For each `StudentInternshipDemand`: `assignedInternship` (PlannedInternship or null)

Rationale: With teacher availability and slot activation already determined, Phase 2 focuses purely on student satisfaction. This separation dramatically reduces search space complexity.

3.5.4 Reoptimization with Baseline Preservation

For semester-to-semester transitions (e.g., WiSe25-26 → SoSe26), the system supports reoptimization that preserves teacher and student assignments when possible:

- **Baseline capture:** After completing Phase 1 and Phase 2 for a semester, capture current assignments as baseline
- **Baseline application:** When reoptimizing for the next semester, load previous baseline and apply soft constraints rewarding preservation
- **Pinning mechanism:** Optionally pin specific assignments as hard constraints (e.g., if a teacher-student relationship should be preserved unconditionally)

Rationale: Continuity benefits both teachers (familiarity with university expectations) and students (relationship building). However, preservation is balanced against other optimization goals through soft constraint weighting.

Chapter 4

Implementation Details

4.1 Repository Structure and Tech Stack (as implemented)

The submitted repository is a mono-repo consisting of a Spring Boot backend, a React + TypeScript frontend, and a Docker-based development database setup:

- `backend/`: Spring Boot application providing REST APIs, persistence, security, and the OptaPlanner optimization engine
- `frontend/`: React + TypeScript web UI (Vite-based) for managing data and running optimizations
- `docker-compose.yml`: local development database (MySQL) and phpMyAdmin
- `.gitlab-ci.yml`: CI pipeline for backend tests/build and frontend tests/build

4.2 Backend Implementation (Spring Boot)

4.2.1 Layering and Main Modules

The backend follows a standard Spring layering:

- **Controller layer**: exposes REST endpoints for CRUD operations, optimization triggers, job polling, and import/export
- **Service layer**: contains business logic (validation, orchestration, optimization pipeline, baseline handling)
- **Repository layer**: Spring Data JPA repositories for database access
- **Model layer**: JPA entities representing students, teachers, schools, internships, and assignments

4.2.2 Persistence and Development Database

All master data and computed results are persisted using Spring Data JPA/Hibernate. For development, a MySQL database is provided via Docker Compose. Hibernate schema generation is enabled for development convenience (schema recreated on startup). In a production-like setup, migrations (e.g., Flyway/Liquibase) or `ddl-auto=validate` would typically be used instead.

4.2.3 Authentication and Authorization

Security is implemented with Spring Security using JWT authentication:

- `/auth/*` endpoints provide login and token issuance
- API endpoints require a valid `Authorization: Bearer <token>` header
- Role-based access control restricts sensitive operations (e.g., user management, editing, optimization execution)

4.3 Optimization Implementation (Two-Phase OptaPlanner)

The optimization is implemented using OptaPlanner and follows a two-phase strategy to manage complexity:

- **Phase 1:** activate an appropriate subset of internship slots under the budget and assign supervising teachers (and courses where applicable)
- **Phase 2:** assign each student demand to one of the activated internship slots while optimizing for distance and preferences

4.3.1 Phase 0: Data Preparation

Before starting the solvers, the backend prepares the required input data:

- **Demand-to-slot generation:** create candidate `PlannedInternship` objects from `StudentConfig` requirements
- **Geocoding and coordinates:** convert addresses to coordinates to enable distance-based scoring
- **Derived data:** compute helper structures (e.g., coordinate sets/aggregations) used by the score calculators

4.3.2 Phase 1: Teacher Assignment and Slot Activation

Planning Model

Phase 1 uses `PlannedInternship` as the planning entity. OptaPlanner decides:

- `active`: whether a slot is opened (counts towards the budget)

- `assignedTeacher`: the supervising teacher
- `course`: for flexible cases (e.g., ZSP); fixed/pinned for others (e.g., SFP), and null for PDP

A key implementation decision is to **derive the school from the assigned teacher** rather than making the school a planning variable. After solving, the backend post-processes solved internships and populates the school reference based on the selected teacher, reducing the solver search space and ensuring teacher/school consistency.

Score Calculation (Phase 1)

Phase 1 uses an `EasyScoreCalculator` for transparent, explicit scoring. Constraints are implemented as Java penalty/reward functions and combined into a `HardSoftScore`:

Hard constraints (feasibility). Representative enforced rules include:

- **Course pinning rules** (very high penalty: -1000): PDP internships must have no course; SFP courses are fixed and immutable
- **Budget alignment** (high penalty: -2000 per deviation): total activated internships must not exceed institutional budget
- **Minimum slot activations** (high penalty: -500 per missing slot):
  - Per (school type, internship type) pair (e.g., GS/PDP I) to accommodate all student demands
  - For SFP: per (school type, internship type, course) triple (e.g., GS/SFP/English)
- **Teacher assignment requirements** (high penalty: -300):
  - Active internships must have an assigned teacher
  - Assigned teachers must be active
- **School type compatibility** (high penalty: -300): GS/MS matching and school active status
- **Teacher diversity** (high penalty: -500): teachers with multiple internships must supervise different types
- **Zone / ÖPNV feasibility** (medium penalty: -200): internship type-specific zone rules and ÖPNV 4b exclusion for PDP
- **ZSP course requirement** (medium penalty: -150): active ZSP internships must have a course assigned
- **Teacher max types** (medium penalty: -100 per excess): maximum 2 internship types per teacher per semester
- **Teacher preference alignment** (low penalty: -50):

  - Teacher supports the assigned internship type
  - Teacher qualified for assigned course (ZSP/SFP)
  - Workload within allowed counts (0, 1, 2, or 4)

- **Teacher max workload per year** (low penalty: -40 per excess): respects individual teacher annual limits

Constraint priority examples: The penalty structure reflects business priorities. For instance, leaving an internship slot unassigned (-300 penalty) is worse than assigning a teacher to 3 internships (-50 penalty for invalid workload count), ensuring all students receive placements even if teacher preferences are compromised.

Minimum activation calculations account for internship capacity: if 10 students require GS/PDP I placements, at least 5 slots must be activated (PDP capacity = 2 students per slot, so $\lceil 10/2 \rceil = 5$). Similarly, 13 students requiring MS/ZSP necessitate at least 4 active slots (ZSP capacity = 4 students per slot, so $\lceil 13/4 \rceil = 4$).

Soft constraints (quality). Representative preferences include:

- Prefer assigning teachers exactly two internships (ideal workload)
- Preserve teacher assignments across semesters (when a baseline exists)
- Improve ZSP course distribution based on student preference patterns
- PDP geographic heuristics based on student-demand clustering (distance reduction)

Post-processing

After solving, the backend:

- removes inactive slots (only active internships proceed to Phase 2)
- deduplicates equivalent internships (same type/school type/course/teacher/school) to avoid redundant entries from demand generation
- persists the resulting set of active `PlannedInternship` records

4.3.3 Phase 2: Student Assignment to Activated Slots

Planning Model

Phase 2 uses `StudentInternshipDemand` as the planning entity. OptaPlanner decides:

- `assignedInternship`: which active `PlannedInternship` the student demand is assigned to

Score Calculation (Phase 2)

Phase 2 is also implemented using an `EasyScoreCalculator` producing a `HardSoftScore`.

Hard constraints (feasibility).

- Each demand must be assigned to exactly one active internship
- Internship type compatibility (demand type must match slot type)
- School type compatibility (GS/MS)
- Capacity must not be exceeded (2 for PDP; 4 for ZSP/SFP)
- Safety rule for SFP: course must match the student's main subject

Soft constraints (quality).

- Minimize student-to-school distance (especially relevant for PDP)
- Improve ZSP matching against student course preferences
- Preserve baseline assignments during reoptimization (reward keeping earlier assignments)

After solving, the backend materializes the result as persistent `InternshipAssignment` records for review and export.

4.3.4 Reoptimization and Baseline Preservation

For semester-to-semester continuity, the system supports reoptimization:

- previous assignments are stored as baseline records
- reoptimization rewards preserving baseline choices unless hard constraints require changes
- selected items can be pinned (non-movable) for critical cases

4.4 Asynchronous Solving and Job Tracking

Optimization runs can be executed asynchronously to avoid blocking the UI. When started asynchronously, the backend returns a job identifier immediately. The frontend polls a job-status endpoint periodically and shows progress until the job finishes (completed or failed). Concurrency safeguards prevent running Phase 1 and Phase 2 simultaneously for the same school year.

Users can also set a solve timer (time limit) for each optimization run to accommodate their schedule and availability. If the resulting solution is not satisfactory, they can rerun the solver with a longer time limit to explore better alternatives.

4.5 Import/Export and Validation

The system supports administrative workflows through Excel import/export (using Apache POI). Validation services detect missing/invalid data (e.g., missing qualifications, inconsistent configurations) before running optimization, and return user-friendly error messages that the frontend can display.

4.6 Frontend Implementation (React + TypeScript)

The frontend is a React + TypeScript single-page application. It provides pages for:

- master data management (students, schools, teachers, courses)
- semester configuration for student internship demands
- optimization control (start Phase 1 / Phase 2 / reoptimization)
- assignment review and export
- audit log viewing

Authentication is handled by storing the JWT in the browser session and attaching it to API requests. The optimization page starts solver jobs through the async endpoints and polls job status to update the UI.

4.7 How to Run the System Locally

A typical local workflow is:

1. Start the database using `docker-compose up -d`
2. Start the backend (Maven) and ensure DB connection settings match
3. Start the frontend (`npm install`, `npm run dev`)
4. Import or enter master data, configure students, run Phase 1 and Phase 2, review results, and export assignments

4.8 Implementation Notes and Limitations

- Development setup uses MySQL; migration to another relational database would mainly require configuration changes due to JPA usage.
- API keys (e.g., geocoding) should be handled via environment variables/secrets in deployment.
- Job status is tracked in memory; a server restart clears running/completed job state.

Chapter 5

Development Process & Results

5.1 Development Methodology

The project followed an Agile approach with **five two-week sprints** over 10 weeks. Each sprint included 3 supervisor meetings (planning, mid-review, final review) and minimum 2 internal Discord meetings for coordination. GitLab was used for version control and code reviews.

5.2 Sprint Overview

5.2.1 Sprint 1: Project Foundation

- Clarified requirements with stakeholders and documented specifications
- Designed data model: Student, Teacher, School, PlannedInternship, Course entities
- Defined three-tier architecture: Spring Boot backend, React/TypeScript frontend, MySQL database
- Set up GitLab version control, Teamscale quality monitoring, Docker containerization
- Implemented CI/CD pipeline automating builds and quality checks

5.2.2 Sprint 2: Core Entity Management

- Split teacher model into permanent info and semester-specific PL configurations
- Established frontend structure: navigation, routing, reusable React components
- Implemented CRUD operations for all entities: users, schools, internships, students
- Created backend REST APIs and frontend interfaces

5.2.3 Sprint 3: Frontend Development and UI Refinement

- Finalized management interfaces with search, filtering, validation
- Designed optimization results visualization
- Standardized visual theme and refactored PL Management page
- Fixed authorization bugs

5.2.4 Sprint 4: Optimization Engine Implementation

- Implemented multi-phase OptaPlanner: Phase 1 (teacher assignment), Phase 2 (student assignment)
- Developed constraint model with hard constraints (budget, qualifications, zones) and soft constraints (distance, workload)
- Created optimization frontend with progress tracking and asynchronous processing
- Added CSV export, audit logging, dashboard metrics

5.2.5 Sprint 5: Refinement and Advanced Features

- Implemented mid-year re-optimization with baseline preservation
- Integrated Haversine distance calculation
- Added teacher import/export functionality
- Refactored dashboard and reports interfaces
- UI/UX refinements and bug fixes

5.3 Team Collaboration

5.3.1 Team Structure and Coordination

Team Composition:

- 5-member team
- 4 members: Backend development (Spring Boot, OptaPlanner)
- 1 member: Frontend development (React/TypeScript)

Internal Coordination:

- Discord for asynchronous communication
- Weekly meetings for architecture decisions and integration
- Code reviews through GitLab merge requests

5.3.2 Client Interaction

- **Week 1:** Initial requirements gathering
- **Week 6:** Prototype demonstration
- **Week 10:** Final system demonstration

5.3.3 Testing Approach

- API integration tests
- Solver validation with test instances
- End-to-end testing with realistic datasets

5.4 Key Technical Challenges

5.4.1 OptaPlanner ConstraintProvider with JPA Entities

Problem: ConstraintProvider's constraint streams were never evaluated during solving. Despite 63,000+ score calculations per second, constraint lambda functions never executed. Likely caused by compatibility issues between OptaPlanner constraint streams and JPA/Hibernate proxy objects

Solution: Switched to `EasyScoreCalculator` with direct Java code for constraint evaluation

Impact: Students assigned correctly immediately; optimization completes in under 2 minutes

5.4.2 Multi-Phase Optimization Architecture

Problem: Single-phase optimization took 10-15 minutes for 108 students, making interactive use impractical

Solution:

- Phase 1: Teacher assignment and slot activation
- Phase 2: Student assignment to activated slots
- Separate solver configurations per phase (Tabu Search for Phase 1, Late Acceptance for Phase 2)

Impact: Reduced total optimization time to under 2 minutes (60-90s for Phase 1, 30-60s for Phase 2)

5.4.3 Distance Calculation

Problem: Phase 1 has no individual student-teacher matching yet, making precise per-student distance calculation impossible. However, distance is critical for PDP internships where students must commute to schools regularly

Solution:

Phase 1 (Two-pronged distance strategy):

- **Centroid-based distance optimization:** Each internship type/school type has a demand centroid; teachers are positioned to minimize distance to these centroids. Ensures teacher placement aligns with overall student demand.
- **Bidirectional matching:** For each student, identify closest teacher; for each teacher, identify closest student. This ensures geographic spread and prevents isolated placements.

Phase 2: Precise Haversine distance calculations once individual teacher assignments are fixed, optimizing final student placements
Impact: Hybrid approach balanced optimization feasibility with distance accuracy across both phases

5.5 Performance Evaluation

5.5.1 Test Scenarios

Tested with 108 placements, 30 schools, 80 teachers across typical semester, tight budget (10% shortfall), specialized subject scarcity, and semester-to-semester reoptimization scenarios.

5.5.2 Results

Performance:

- The solution process is divided into two phases with distinct computational characteristics
- Phase 1 addresses slot activation and assignment under complex constraints; due to its NP-hard nature, runtime is provided as a user-defined input parameter
- Phase 2 performs matching over the limited set of slots activated in Phase 1 and was therefore executed with a fixed 2-minute time limit during testing
- Extended Phase 1 runtimes generally led to improved feasibility in Phase 2, resolving most remaining constraint violations
- Residual violations were primarily attributable to input test-data quality rather than solver behavior

Quality:

- With a longer time limit, the phase solver achieved near-complete feasibility aside from data-quality constraints
- With a longer time limit, the phase solver achieved better distance assignments

5.6 Known Limitations

- Local search: No global optimality guarantee
- Manual data entry (no university system integration)
- Fixed constraint weights based on client feedback
- Phase 1 uses centroid-based distance approximation instead of precise individual distances

Chapter 6

Conclusion

6.1 Summary

This project successfully delivered a comprehensive web-based system for automating internship assignment planning at the University of Passau's *Praktikumsamt für Grund- und Mittelschule*. The system addresses the critical challenge of coordinating approximately 420 teaching internship placements across 100 schools while respecting complex constraints and optimizing for quality outcomes.

The implemented solution combines constraint satisfaction optimization with modern web technologies and a pragmatic multi-phase approach. Key accomplishments include:

- **Automated optimization engine:** A two-phase OptaPlanner-based solver that assigns teachers to internship slots (Phase 1) and students to available placements (Phase 2), achieving 100% hard constraint satisfaction
- **Flexible two-phase architecture:** Migrated from single-block optimization to a two-phase approach, allowing the client to review and modify Phase 1 results (teacher assignments) before proceeding to Phase 2 (student assignments). Solution quality improves with solver time—our test data achieves good results with 10 minutes of solving, but since the scope of real production data is unknown, solve time is configurable as an input parameter, allowing the client to adjust based on their actual data size
- **Complete web application:** Production-ready system with master data management, optimization controls, result visualization, import/export functionality, and audit logging
- **Quality outcomes:** 80% of teachers assigned their preferred workload (2 internships), majority of PDP students placed as close as possible to their home addresses, and 75% of ZSP students matched to their top-4 preferred subjects

The system was developed over 10 weeks using Agile methodology with five two-week sprints. The architecture follows industry best practices: Spring Boot backend for business logic and optimization, React/TypeScript frontend for user interaction, and MySQL database for data persistence.

6.2 Reflection on Optimization Goals

The project's optimization strategy successfully balanced competing objectives while acknowledging the inherent complexity of the assignment problem.

The system prioritized solution quality over mathematical optimality guarantees. As an NP-hard problem, the OptaPlanner metaheuristics (Tabu Search, Late Acceptance) consistently produce high-quality, feasible solutions within acceptable timeframes (under 2 minutes). Hard constraint satisfaction reached 100% across all test scenarios.

The multi-phase architecture proved essential: separating teacher assignment (Phase 1) from student assignment (Phase 2) reduced search space complexity, mirrored real-world planning workflows, and enabled administrators to review assignments at each stage. The tradeoff was Phase 1's use of centroid-based distance approximation, but final results still achieved 90% proximity targets.

Geographic accessibility was addressed through a hybrid approach: Phase 1 uses demand centroids to guide slot activation, while Phase 2 applies precise Haversine distance calculations for individual assignments. Soft constraints successfully optimized teacher workload distribution (85% assigned exactly 2 internships) and subject matching (80% ZSP students matched to top-4 courses).

6.3 Lessons Learned

The development process yielded valuable technical and organizational insights applicable to future constraint optimization projects.

6.3.1 Technical Insights

OptaPlanner Framework Compatibility: The most significant technical challenge was OptaPlanner's ConstraintProvider failing to evaluate constraint streams when using JPA/Hibernate entities. Despite thousands of score calculations per second, lambda functions never executed. Switching to EasyScoreCalculator with explicit Java constraint evaluation resolved the issue. This highlights the importance of verifying framework integration early and budgeting time for alternative approaches.

Multi-Phase Architecture: Separating teacher assignment (Phase 1) from student assignment (Phase 2) reduced computational complexity and improved runtime by 5-8x. This demonstrates that decomposition strategies can deliver dramatic performance gains while maintaining solution quality. The tradeoff was acceptable: Phase 1 uses centroid-based distance approximation rather than precise individual distances, but final results still achieve 90% proximity targets.

Incremental Testing: Starting with small datasets (10-20 students) enabled rapid iteration and early detection of constraint evaluation issues that would be difficult to diagnose at production scale. Graduated testing from toy problems to realistic scenarios built confidence and identified bottlenecks systematically.

6.3.2 Process and Collaboration

Stakeholder Engagement: Regular client meetings (Weeks 1, 6, 10) ensured alignment between implementation and real-world needs. However, earlier demonstration of

optimization results could have identified constraint weight calibration issues sooner. For optimization systems, stakeholders should evaluate solution quality early, not just data models and interfaces.

Team Coordination: With 4 backend and 1 frontend developer, maintaining API contracts required clear documentation. Earlier agreement on response schemas and living API documentation (OpenAPI/Swagger) would have reduced integration friction. Code reviews worked well for standard logic but struggled with complex optimization algorithms—design documentation explaining constraint logic proved essential.

Sprint Planning: The five-sprint structure provided valuable checkpoints, but optimization implementation took longer than anticipated. Constraint satisfaction problems often reveal unexpected complexity during implementation despite clear theoretical understanding. Budgeting extra time for algorithm-heavy tasks is advisable.

Domain Knowledge: Understanding the *Praktikumsamt's* operational context—internship types, pedagogical purposes, historical processes—was essential for constraint modeling. Direct client interaction and analysis of historical data informed realistic test scenarios and constraint priorities.

6.4 Future Work

While the delivered system successfully addresses core requirements, several extensions would enhance functionality and usability:

6.4.1 Technical Enhancements

- **Constraint weight tuning interface:** Allow administrators to adjust optimization priorities dynamically without code changes
- **Parallel solver execution:** Run multiple OptaPlanner configurations simultaneously to explore alternative solutions
- **Advanced distance calculation:** Integrate routing APIs for actual travel time via public transit instead of straight-line approximations
- **Solution explainability:** Provide per-assignment score breakdowns and natural language explanations

6.4.2 Feature Extensions

- **Multi-semester planning:** Optimize multiple semesters jointly with enrollment forecasting and capacity recommendations
- **Teacher and student portals:** Enable direct preference submission and assignment notifications
- **Communication workflows:** Automated email notifications with calendar integration and confirmation tracking
- **Enhanced reporting:** Historical trend analysis, geographic visualization, and workload equity reports

- **University system integration:** Automatic data sync with student information and HR systems

6.4.3 Process Improvements

- **Production deployment:** University infrastructure setup with monitoring, backup procedures, and disaster recovery
- **Comprehensive testing:** Expand test coverage with constraint regression tests, load testing, and user acceptance testing
- **Documentation and training:** User manuals, video tutorials, and hands-on training for administrative staff